Git & GitHub for Research Projects

A Practical Workbook Using VS Code

From Working Alone to Collaborative Research

Purpose of this workbook. This document is designed to be used while working, not simply read. The goal is to understand how Git and GitHub work in an individual research project, and then learn how the same project changes when more than one person contributes.

Índice

1. How to Use This Workbook

This workbook follows one project through three stages:

1. **Git for individual work**: one researcher, one computer, one repository.
2. **Transition to collaboration**: understanding what changes when another person joins.
3. **Collaborative Git**: branches, pull requests, reviews, merges, conflicts, and team rules.

The recommended approach is:

- read the short explanation;
- complete the action in VS Code or GitHub;
- check what happened;
- record questions or problems;
- repeat until the workflow feels natural.

Working rule: do not try to memorise Git commands. First understand the workflow. Commands and buttons become much easier once the logic is clear.

2. The Research Project We Will Use

Throughout the workbook, imagine a standard research project with the following structure:

```
research-project/
|
|-- code/
| |-- 00_master.do
| |-- 01_clean.do
| |-- 02_analysis.do
|
|-- data_raw/
|-- data_processed/
|
|-- output/
| |-- tables/
| |-- figures/
|
|-- docs/
|
|-- README.md
|-- .gitignore
```

The same logic applies to projects using Stata, R, Python, LaTeX, or combinations of these tools.

GitHub should not automatically be treated as data storage. Confidential data, restricted microdata, personal identifiers, credentials, and very large raw datasets should generally remain outside the repository.

3. Part I: Git for Individual Research

3.1 What Git Is

Git is a version-control system. It records the history of a project.

A useful mental model is:

Edit files → Review changes → Commit → Push to GitHub

A **commit** is a saved checkpoint in the history of the project.

GitHub is the online location where the repository can be stored and shared.

3.2 Local Repository vs Remote Repository

Local repository	The version stored on your computer. You normally edit these files in VS Code.
Remote repository	The version stored on GitHub.
Commit	A recorded checkpoint in the project's history.
Push	Send local commits to GitHub.
Pull	Bring changes from GitHub to the local computer.

3.3 Exercise 1: Open a Project in VS Code

Task

Open VS Code.
Open a local research-project folder.
Identify the Explorer panel.
Identify the Source Control panel.
Open the terminal inside VS Code.

Notes:

3.4 Exercise 2: Check Whether a Folder Is a Git Repository

Open the VS Code terminal and run:

```
git status
```

If Git is active, the terminal will show information about the current branch and file status.

If Git is not active, initialise the repository with:

```
git init
```

Check

I can run `git status`.
I know which branch I am on.
I understand whether my files are tracked or untracked.

3.5 Tracked, Modified, and Untracked Files

Git classifies files according to their status.

- **Untracked**: Git has not started tracking the file.
- **Modified**: a tracked file has changed.
- **Staged**: the change has been selected for the next commit.
- **Committed**: the change has been stored in Git history.

3.6 Exercise 3: Make the First Commit

Modify `README.md`.

In VS Code Source Control:

1. inspect the changed file;
2. stage the change;
3. write a commit message;
4. commit.

Example:

```
Add initial project description
```

Then check:

```
git status
```

A good commit is a small, meaningful unit of work. A commit should ideally answer: *what changed?*

3.7 Good and Bad Commit Messages

Better	Avoid
Add project README	changes
Create master do-file	update
Fix variable labels	final
Add Table 1 export	final2
Document data source	now final

3.8 Connecting the Project to GitHub

A project can be created on GitHub and then cloned, or an existing local project can be connected to a remote repository.

The important concept is:

Local repository $\longleftrightarrow$ Remote GitHub repository

To inspect the remote connection:

```
git remote -v
```

3.9 Push and Pull

Push sends commits from your local repository to GitHub.

```
git push
```

Pull downloads and integrates changes from GitHub.

```
git pull
```

Exercise 4

Make a small change locally.
Commit it.
Push it.
Open GitHub in the browser.
Confirm that the change appears online.

3.10 The `.gitignore` File

The `.gitignore` file tells Git which files or folders should not be tracked.

Example:

```
# Data
data_raw/*
data_processed/*

# Temporary files
*.tmp
*.log

# System files
.DS_Store

# Stata temporary files
*.dta~
```

If empty folders need to remain visible in GitHub, place a small placeholder file such as `.gitkeep` inside them.

Do not use `.gitignore` as a substitute for data security. If confidential data have already been committed, simply adding the file to `.gitignore` does not remove the data from Git history.

4. Part II: From Working Alone to Working With Someone

4.1 What Changes?

When working alone, a simple workflow can be sufficient:

Edit → Commit → Push

When another person joins, the workflow changes:

Update local project → Create branch → Work → Commit → Push → Pull Request → Review → Merge

4.2 Why Not Everyone on `main`?

The `main` branch should represent the stable version of the project.

If several people edit directly on `main`:

- unfinished work can affect everyone;
- review becomes difficult;
- mistakes are harder to isolate;
- parallel work becomes confusing.

The solution is to work in **branches**.

4.3 Understanding Branches

A branch is a separate line of development.

```
main
 |
 |------ feature/update-readme
 |
 |------ feature/clean-data
 |
 |------ feature/table-formatting
```

Each person can work without immediately changing `main`.

4.4 Working Alone vs Working Collaboratively

Task	Working alone	Working collaboratively
Start working	Open project	Pull latest version first
Edit code	Often directly on main	Create or switch to task branch

Save progress	Commit	Commit
Share progress	Push	Push branch
Review	Usually self-review	Pull Request + review
Finish task	Continue on main	Merge approved branch into main
Start next task	Continue	Update main and create a new branch

The major conceptual change is this: in a team, Git is not only a version history. It is also a system for separating, reviewing, and integrating work.

5. Part III: Collaborative Git in VS Code

5.1 The Standard Team Workflow

A simple collaborative workflow is:

1. update `main`;
2. create a branch;
3. make changes;
4. review local changes;
5. commit;
6. push the branch;
7. open a Pull Request;
8. review the Pull Request;
9. make corrections if required;
10. merge into `main`;
11. delete or stop using the completed branch;
12. update local `main` again.

5.2 Exercise 5: Create a Branch

First make sure you are on `main`:

```
git switch main
```

Update it:

```
git pull
```

Create and switch to a branch:

```
git switch -c feature/update-readme
```

Check:

```
git branch
```

The active branch will be marked with an asterisk.

Check

I know which branch is active.
I can create a branch.
I understand that the branch starts from the version of `main` that existed when I created it.

5.3 Exercise 6: Work and Commit on the Branch

Make a small change to `README.md`.

Then:

```
git status
```

Stage and commit using VS Code Source Control or the terminal.

Example:

```
git add README.md
git commit -m "Clarify project folder structure"
```

5.4 Exercise 7: Push the Branch

The first push may require:

```
git push -u origin feature/update-readme
```

Afterward, regular pushes can normally use:

```
git push
```

5.5 Pull Requests

A Pull Request (PR) asks to merge one branch into another.

Normally:

`feature/update-readme` → `main`

A useful PR should explain:

- what changed;
- why it changed;
- what files were affected;
- anything the reviewer should check.

Example:

```
Title:
Clarify README and project folder structure

Description:
- Added explanation of raw and processed data folders
- Clarified output directory
- No analysis code changed
```

5.6 Exercise 8: Open a Pull Request

On GitHub:

Open the repository.

Select the pushed branch.

Create a Pull Request into `main`.

Add a clear title.

Add a short description.

Review the changed files.

5.7 Reviewing a Pull Request

Before merging, the reviewer should inspect:

- the Files Changed tab;
- unexpected files;
- large deletions;
- code quality;
- naming consistency;
- whether restricted data were accidentally added;
- whether the task requested was actually completed.

The reviewer can:

- approve;
- comment;
- request changes.

5.8 Exercise 9: Request a Change

Before merging a practice PR:

Leave one review comment.

Make an additional change on the branch.

Commit it.

Push again.

Confirm that the Pull Request updates automatically.

A Pull Request is not a static document. It represents the current state of the branch. New commits pushed to that branch appear in the same PR.

5.9 Merging

Once the PR is ready, merge it into `main`.

Then return to VS Code:

```
git switch main
git pull
```

Your local `main` now contains the merged work.

End-to-end practice complete

Updated main.
Created branch.
Edited file.
Committed.
Pushed.
Opened PR.
Reviewed PR.
Merged.
Pulled updated main locally.

6. Part IV: Simulating Two People Working Together

6.1 Scenario A: Different Files

Person A edits:

```
docs/README_data.md
```

Person B edits:

```
code/02_analysis.do
```

Both use separate branches.

This is usually straightforward because Git can combine independent changes easily.

6.2 Scenario B: The Same File

Person A changes line 20 of:

```
code/02_analysis.do
```

Person B also changes line 20.

Git may not know which version should win.

This is a **merge conflict**.

6.3 What a Merge Conflict Looks Like

Git may display something similar to:

```
<<<<<<< HEAD
reg outcome treatment controls
=======
reg outcome treatment controls i.year
>>>>>>> feature/add-year-fe
```

The researcher must decide the correct final version and remove the conflict markers.

> Do not choose "Accept Current" or "Accept Incoming" automatically. First understand what each version contains and what the intended analysis should be.

6.4 Exercise 10: Create a Small Conflict on Purpose

Practice conflicts in a disposable repository.

1. Create two branches from the same version of `main`.
2. Change the same sentence in both branches.
3. Merge one branch.
4. Try to merge the second.
5. Inspect the conflict in VS Code.
6. Resolve it manually.

Notes:

7. Part V: A Practical Research-Team Workflow

7.1 Recommended Branch Structure

For small research teams, branch names should describe tasks rather than people.

Examples:

```
feature/add-data-readme
feature/clean-labour-data
feature/format-table1
fix/variable-labels
docs/update-methodology-notes
```

This keeps the project understandable even after team members change.

7.2 Recommended Commit Style

Use short action-oriented messages:

```
Add labour-market variable labels
Clean institution identifiers
Export descriptive statistics table
Update README with data source
Fix missing-value handling
```

7.3 Recommended Pull Request Template

```
## What changed?

## Files changed

## Checks completed
- [ ] Code runs
- [ ] No restricted data added
- [ ] Outputs checked
- [ ] README/documentation updated if necessary

## Notes for reviewer
```

7.4 Before Starting a New Task

Switch to `main`.
Pull the latest changes.
Confirm the working tree is clean.
Create a new branch for the task.
Confirm the correct branch is active.

7.5 Before Opening a Pull Request

Check `git status`.
Review all changed files.
Remove temporary files.
Run the relevant code.
Check outputs.
Write clear commit messages.
Push the branch.
Open the PR with a short explanation.

8. Part VI: Research Files and Data

8.1 What Usually Belongs in GitHub

Good candidates include:

- Stata do-files;
- R scripts;
- Python scripts;
- LaTeX files;
- README files;
- data dictionaries;
- documentation;
- small, non-sensitive configuration files;
- selected tables and figures when useful.

8.2 What Usually Does Not Belong in GitHub

- confidential microdata;
- restricted-access datasets;
- identifying information;
- passwords, tokens, API keys;
- very large raw datasets;
- unnecessary temporary files;
- software-generated caches.

8.3 GitHub + External Data Storage

A common research setup is:

GitHub: code + documentation + version control
Drive/OneDrive/secure server: raw and restricted data

The README should explain where the external data are expected to be placed.

9. Part VII: Common Problems

9.1 I Edited on `main` by Accident

Do not panic.

First check:

```
git status
```

If the changes have not been committed, they can often be moved to a new branch:

```
git switch -c feature/my-task
```

Then commit normally.

9.2 I Forgot to Pull Before Starting

If nobody changed the same files, the situation may be easy to resolve.

If Git reports conflicts, inspect them carefully before proceeding.

9.3 I Pushed the Wrong File

Do not simply delete it from the current folder and assume the issue is solved.

If the file contains sensitive information, treat this as a security issue because Git history may still contain it.

9.4 My Branch Is Out of Date

One simple workflow for beginners is:

```
git switch main
git pull
git switch feature/my-task
git merge main
```

Resolve conflicts if necessary, then continue.

9.5 I Do Not Know What Git Is Doing

Use:

```
git status
```

This should be the first diagnostic command.

Also useful:

```
git branch
git log --oneline --graph --decorate --all
git remote -v
```

10. Part VIII: VS Code Reference

The most useful VS Code areas for Git work are:

- **Explorer**: files and folders.
- **Source Control**: changed files, staging, commit.
- **Integrated Terminal**: Git commands.
- **Branch indicator**: current branch.
- **Diff view**: compare old and new versions.

10.1 Terminal Commands to Know

Command	Meaning
`git status`	Show current state of repository
`git branch`	Show branches
`git switch main`	Move to main branch
`git switch -c branch-name`	Create and enter a branch
`git pull`	Download and integrate remote changes
`git add file`	Stage a file
`git commit -m "message"`	Create a commit
`git push`	Upload commits
`git log -oneline`	Show compact commit history
`git remote -v`	Show GitHub remote connection

The commands are useful, but VS Code can perform many of these operations through its interface. The important skill is understanding what operation is taking place.

11. Part IX: Practice Sessions

Session 1: Individual Git

Open project in VS Code.

Identify local repository.

Edit README.

Stage change.

Commit.

Push.

Confirm change on GitHub.

Session 2: First Branch

Pull latest main.

Create branch.

Modify documentation.

Commit.

Push branch.

Open PR.

Merge.

Pull updated main.

Session 3: Two-Person Simulation

Create two task branches.

Make independent changes.

Open two PRs.

Merge first PR.

Update second branch if needed.

Merge second PR.

Session 4: Conflict Practice

Make two branches.

Edit same line.

Merge first branch.

Attempt second merge.

Resolve conflict in VS Code.

Confirm final file.

Session 5: Convert a Real Research Project

Check folder structure.

Create or improve README.

Create `.gitignore`.

Identify files that should not be tracked.

Confirm data-storage strategy.

Define branch rules.

Define PR rules.

Complete one real task through the full workflow.

12. Part X: Personal Notes and Questions

Things I Understand Well

Things I Still Find Confusing

Questions to Test Next Time

13. One-Page Workflow Reference

Working alone
Open project → edit → review changes → commit → push

Working collaboratively
`main` → pull → create branch → work → commit → push → Pull Request → review → merge → pull updated `main`

Before every new collaborative task:
`git switch main`
`git pull`
Create a new branch.
Do not reuse an old task branch for unrelated work.

Never commit: restricted data, passwords, credentials, personal identifiers, or files you do not understand.

The goal is not to become a Git specialist. The goal is to make research work traceable, reviewable, reproducible, and safe to collaborate on.